

IDV-G4ME — LABYRINTHE

LABYRIN X



LABYRIN

X

Balise. Switch. Dash. Évasion.

IDV-G4ME — LABYRINTHE	1
Contexte du projet	3
Présentation du jeu	3
Objectifs pédagogiques	3
Contraintes techniques	3
Vue globale	4
Spécifiques « Labyrinthe »	4
Fonctionnalités	4
Présentation des niveaux	4
Graphiques & audio	5
Outils de debug	5
Architecture du projet	5
Bonnes pratiques & conventions de code	5
Outils de travail	6
Rendus & livrables	6

Contexte du projet

Ce projet s'inscrit dans une initiation structurée au moteur Unity. L'objectif est d'apprendre à concevoir un jeu 3D simple, clair et jouable, en découvrant par la pratique la construction de scènes, la manipulation d'objets (GameObjects & Components), l'écriture de scripts C#, la physique, l'UI/UX, etc. Nous avons donc retenu le format labyrinthe.

Présentation du jeu

Titre du projet : LABYRINX

Pitch du jeu : Le joueur doit s'extraire d'un complexe modulable en un temps limité, en activant des interrupteurs, en évitant pièges et ennemis et en optimisant sa trajectoire. La fin de niveau attribue 1 à 3 étoiles selon le temps, avec un classement local.

Type de caméra : TPS (vue troisième personne), avec rotation à la souris. Ce choix maximise la lisibilité des espaces, des pièges et des ennemis.

Plateformes : PC/Linux/Max

Nombre de joueurs : 1 joueur.

Rythme & durée : 2 à 3 niveaux, 2 à 3 minutes par niveau.

Objectifs pédagogiques

Au terme du projet, l'étudiant est capable de :

- Comprendre l'éditeur Unity et organiser une scène.
- Manipuler GameObjects/Components (colliders, triggers, CharacterController, materials, prefabs).
- Programmer en C# des mécaniques de gameplay, des interactions, une IA simple, un timer, et une persistance locale.
- Concevoir une UI/UX lisible (HUD, menus, options, fin de partie) avec des feedbacks visuels/sonores.
- Orchestrer plusieurs scènes et exporter un build jouable.
- Collaborer via GitLab (branches, MR, Tag Gold) avec des conventions de code homogènes.

Contraintes techniques

- **Rendu** : 3D.
- **Langage** : C# uniquement (pas de Visual Scripting).
- **Contrôles** : clavier obligatoire.
- **Build** : PC (.exe).

Vue globale

- **Menu principal** : Une scène dédiée donne accès à *Jouer*, *Scores*, *Options*, *Quitter*, *Scores*. Un tableau des meilleurs temps (top 10 local par niveau) est consultable depuis le menu principal ; les scores sont enregistrés en local.
- **Options** : Disponible depuis le menu, ou en jeu. Donne accès au volume du jeu, au niveau de difficulté et à la sensibilité de la caméra.
- **Scènes de jeu** : 2 à 3 scènes de niveaux jouables, plus le menu principal.
- **Animations** : Transitions de scènes, animation de portes, mouvements de pièges ; micro-animations UI pour le feedback.
- **Menu Pause réel** : La pause fige le jeu, et affiche PAUSE au milieu de l'écran.
- **Fin de partie** : Un écran affiche le temps, le nombre d'étoiles et les actions actions (Rejouer / Suivant / Menu).
- **UI** : Le temps de jeu est toujours visible.

Spécificités du format Labyrinthe

- 3 niveaux de 2 à 3 min.
- Pièges + ennemis : piques, trous dans le sol, murs qui se referment ; ennemis mobiles (chauves-souris, araignées, squelettes...).
- Interrupteurs : des interrupteurs de type switch permettent d'ouvrir des portes avec un feedback visuel et/ou sonore.
- Repérage : possibilité de se repérer via des tags sur les murs pour marquer son chemin.
- Timer permanent + étoiles : timer affiché en continu ; à l'arrivée, attribution de 1 à 3 étoiles selon des seuils par niveau, puis enregistrement du temps au classement.

Fonctionnalités

Contrôles au clavier :

- Déplacement : ZQSD ; Shift courir ; Espace saut (unique) ; E dash ; clic gauche attaque CàC ; souris X/Y rotation ; R déposer une balise ; Échap pause.
- Physique & sécurité : Gravité crédible, collisions stables, réapparition en cas de chute/mort (spawn/checkpoint).
- Interactions : Interrupteurs actionnables clairement identifiés ; zone de fin détectée pour valider la réussite et déclencher l'écran de score.

Présentation des niveaux

- **Niveau 1** : Un interrupteur, une zone avec piques, un ennemi (chauves-souris). Seuils étoiles : ★★★ ≤ 1:30 ; ★★ ≤ 2:15 ; ★ ≤ 3:00.
- **Niveau 2** : Deux interrupteurs, piques + trous, deux ennemis (chauves-souris + squelettes). *Seuils* : ★★★ ≤ 1:45 ; ★★ ≤ 2:30 ; ★ ≤ 3:00.
- **Niveau 3** : Trois interrupteurs + portes, ennemis précédents + murs qui se referment. *Seuils* : ★★★ ≤ 2:00 ; ★★ ≤ 2:45 ; ★ ≤ 3:15.

Graphiques & audio

Graphiques.

- **Joueur** : un modèle 3D (capsule tolérée au début, modèle humanoïde low-poly recommandé).
- **Labyrinthe, pièges, ennemis** : modèles 3D différenciés pour éviter toute ambiguïté visuelle.
- Audio facultatif: son lorsqu'un niveau est terminé, son sur le menu principal, son d'ambiance.
- SFX : saut, collision, activation interrupteur, ouverture porte, fin de niveau.
- Musique de fond discrète, non intrusive.

Outils de debug

- Invincibilité : touche I .
- Changement de niveau : touches ctrl + 1 / 2 / 3.

Bonnes pratiques & conventions de code

- Langage unique (FR) pour le code et les commits.
- Commentaires utiles et concis.
- Nommage homogène : Classes/Méthodes en PascalCase, variables/champs en camelCase.
- Qualité : compilation propre, warnings traités, PR/MR relues, README et docs mis à jour.

Outils de travail

- Développement : Unity (3D), IDE Visual Studio / VS Code.
- Organisation : tickets sur gitLab.
- Versionning : GitLab, avec les branches et les commits/MR, Tag Gold pour la version finale.

Rendus & livrables

- Cahier des charges (ce document, PDF).
- Document d'architecture (Excel/Google Sheets : Assets, Scenes, Prefabs, Scripts).
- UML (use cases, classes, états IA).
- Build final du jeu (PC).
- Présentation (PowerPoint/Slides/Canva).
- README (≤ 3 pages) : gameplay, assets & licences, usage d'IA générative, fonctionnement global.
- Tag Gold sur GitLab.
- Vidéo de démonstration (1–2 min).
- 5 images “coup de cœur”.